

BORDER OPTICS — POORI RESEARCH SAMJHO

PDF 12 of 22 — Core Analysis: H4 DiD Model

**did_model.py — Poori Line By Line, Control-Contamination
Reversal Ke Saath**

Sakshi ke liye, Sakshi ki apni research par. PDF 11 mein humne dekha ki H1 test (treated villages, akele, before-vs-after) ka answer null hai. Ab is PDF mein H4 test aata hai — treated villages ko akele nahi, balki 721 'control' villages (jo VVP-I ke under nahi aate) ke against compare karke poochna 'kya treated villages control villages se zyada badle?' Ye is poore study ka doosra bada reversal chhupaaye baithi file hai — ek control-group contamination bug jiski wajah se ek significant result asli nahi tha.

did_model.py ek Difference-in-Differences (DiD) model chalata hai — PDF 4 mein humne DiD ka concept seekha tha, ab yahan wahi concept, real code aur real, kabhi-galat-kabhi-sahi numbers ke saath milta hai.

Shuru Karne Se Pehle — H1 Se H4 Tak Ka Pul

PDF 11 khatam hua tha is baat par ki `analyze_results.py` ka H1 test — treated villages, akele, apna hi 2021-vs-2025 before/after — har combination mein null nikla. Lekin ek akela before/after comparison mein ek chhupi hui kamzori hai: agar poore desh mein hi, VVP-I se bilkul unrelated kisi wajah se (jaise ek naya national satellite-imagery-quality improvement, ya ek general economic trend), saari jagah ka NDBI 2021 se 2025 tak thoda badal gaya ho, toh treated villages ka 'before vs after' badlaav us general trend ko bhi pakad lega — aur hum galti se usko VVP-I ka effect keh denge.

Isi kamzori ko theek karne ke liye H4 hypothesis hai — aur DiD (Difference-in-Differences) design. PDF 4 mein humne ye seekha tha ki DiD ka poora point ye hai: sirf treated villages ka before-vs-after mat dekho, ek 'control group' bhi rakho — VVP-I ke bahar wale, lekin similar villages — aur unka bhi wahi before-vs-after nikaalo. Agar treated group control group se genuinely zyada badla, tabhi VVP-I ka asli effect maana jaayega — kyunki jo bhi general, poore-desh-wala trend tha, wo dono groups mein equally aaya hoga, aur 'difference of differences' lete waqt cancel ho jaayega.

`did_model.py` yahi DiD model implement karta hai — 251 treated villages vs 721 'non-VVP, district-restricted' control villages (control-village selection ki apni poori kahani `select_control_villages.py` mein hai, PDF 7 mein cover ho chuki). Lekin is file ki apni bhi ek badi history hai: is control-group mein kabhi ek contamination bug tha — 20 control villages asal mein treated villages hi thay, sirf alag naam/script ke saath double-entry ki wajah se control list mein bhi aa gaye thay. Jab tak wo bug tha, is study ka ek significant result tha (full-year night-lights DiD, $p=0.0364$). Bug fix hone ke baad, wahi result null ho gaya. Ye poori kahani, exact us jagah pe jahan control-data code mein aata hai, aage detail mein aayegi.

Part A — Setup: Docstring, Imports, Path Dictionaries

Docstring — File Ka One-Line Summary

```
"""
DiD: 251 treated villages vs 721 district-restricted non-VVP control villages
(dedup + district-boundary verification logic is in select_control_villages.py). District FE,
SEs clustered by district.
"""
```

Purpose: Ye docstring, sirf do line mein, is file ki poori identity bata deta hai — kitne treated (251), kitne control (721), control kaise chuna gaya (dusri file mein), aur model

kis tarah ka hai (District Fixed Effects — 'FE' — aur cluster-robust standard errors, district ke hisaab se). '721' number apne aap mein ek chhoti history-note hai — agar Sakshi purane commits ya purani PDF dekhe toh '732' ya '735' bhi mil sakta hai, kyunki ye number teen baar badla hai. Wo poori kahani aage aayegi. Syntax: Multi-line docstring, poore project mein consistent style.

Imports — Ek Naya Statistical Library: statsmodels

```
import argparse
import json

import pandas as pd
import statsmodels.formula.api as smf
from scipy import stats
```

Purpose: argparse aur json Python ki built-in libraries hain — pehla command-line se --window jaisa flag padhne ke liye (PDF 11 mein dekha), doosra Python dictionary ko disk par .json file ki tarah save/load karne ke liye. pandas familiar hai. scipy.stats bhi familiar hai (Wilcoxon test PDF 11 mein dekha tha) — yahan Mann-Whitney U test ke liye use hoga, jo is PDF mein pehli baar aayega. Sabse naya hai statsmodels — ye Python ki sabse standard regression-modeling library hai, jo economists/statisticians 'proper' regression models (OLS, logistic, panel-data models, sab kuch) fit karne ke liye use karte hain. formula.api as smf uska ek khaas hissa hai jo R programming language jaisa 'formula string' syntax deta hai — jaise "y ~ x1 + x2" likh kar bolna 'y ko x1 aur x2 se predict karo' — bina khud matrix-algebra likhe. Syntax: import x.y.z as short_name — deeply-nested module ko chhote naam se import karna. from package import thing — package ke andar se ek specific cheez seedhe import karna.

EXAMPLE — R-Style Formula String Kya Hoti Hai

Agar hum chaahte hain ki NDBI-change ko 'treatment', 'post', aur unke interaction se predict kare, toh manually likhna padta: ek matrix banao jisme har column ek predictor ho, phir usko linear-algebra formula (coefficients = $(X'X)^{-1} X'y$) se solve karo. statsmodels.formula.api ye sab chhupa deta hai — sirf ek string likho, "ndbi ~ treatment + post + did_term + C(district)", aur library khud samajh jaati hai ki 'ndbi' outcome hai (bayen taraf), baaki sab predictors hain (dayen taraf), aur C(district) ka matlab hai 'district ko ek categorical variable ki tarah treat karo' (jaise agar district 14 alag-alag values le sakta hai, toh usko 13 alag 0/1 dummy-columns mein tod do — isi ko 'fixed effects' kehte hain, jo aage detail mein aayega).

Path Dictionaries — Ab Panch Dictionaries Ek Saath

```
TREATED_PATHS = {
    "full_year": "data/processed/border_optics_village_results_analyzed.csv",
    "summer": "data/processed/border_optics_village_results_summer_analyzed.csv",
}
CONTROL_PATHS = {
    "full_year": "data/processed/border_optics_control_results.csv",
```

```

    "summer": "data/processed/border_optics_control_results_summer.csv",
}
PANEL_OUT = {
    "full_year": "data/processed/border_optics_did_panel_fullyear.csv",
    "summer": "data/processed/border_optics_did_panel_summer.csv",
}
SUMMARY_OUT = {
    "full_year": "data/processed/border_optics_did_summary_fullyear.json",
    "summer": "data/processed/border_optics_did_summary_summer.json",
}
DISTRICT_OUT = {
    "full_year": "data/processed/border_optics_did_by_district_fullyear.csv",
    "summer": "data/processed/border_optics_did_by_district_summer.csv",
}

```

Purpose: PDF 11 mein humne ek hi path-dictionary pattern dekha tha (input/output, do dictionaries). Yahan wahi pattern paanch dictionaries tak badh gaya hai, kyunki is file mein paanch alag cheezein hain jo --window ke hisaab se badalti hain — treated data kahan se aaye, control data kahan se aaye, banaaya hua 'panel' (aage samjhaayenge) kahan save ho, JSON summary kahan save ho, aur per-district breakdown CSV kahan save ho. Ye isi liye zaroori hai kyunki agar in mein se koi bhi path hardcoded hota, aur kal kisi ne --window summer chalaaya hota galti se full-year ki file ke saath, toh exactly wahi 'window-consistency bug' aata jo PDF 11 ke analyze_results.py ke docstring mein already discuss ho chuka hai. Ek dictionary-per-path pattern is bug ko structurally impossible bana deta hai. Syntax: Paanch alag dictionaries, sabka structure same — key hamesha "full_year"/"summer" hai.

Part B — build_panel(): Wide Se Long Panel Banana

Ye function is poore file ka sabse conceptually-important hissa hai — isse pehle ki DiD model chal sake, data ko ek khaas shape mein laana padta hai, jise 'long panel' kehte hain. Ye shape samajhna DiD samajhne jitna hi zaroori hai, isliye ismein thoda time lagayenge.

```

def build_panel(window):
    treated = pd.read_csv(TREATED_PATHS[window])
    # core sample only - matches control group's scope (Himachal has no control match)
    treated = treated[treated["is_core_sample"] == True].copy()
    treated["treatment"] = 1

    control = pd.read_csv(CONTROL_PATHS[window])
    control["treatment"] = 0

```

Purpose: Treated data padhi jaati hai, sirf core-sample (251, Himachal ki 7 illustrative villages chhod kar — kyunki control group sirf Arunachal, Sikkim, Uttarakhand ke liye bana hai, Himachal ka koi control match nahi hai). Fir ek naya column "treatment" banaaya jaata hai, sabke liye 1 — matlab 'ye group treated hai'. Control data padhi jaati hai, usko treatment = 0 mil jaata hai. Yahi wo do lines hain jahan us control-CSV se data is file mein aata hai — `border_optics_control_results.csv` — jiski apni ek badi history hai, thodi der mein aayegi. Syntax: `df[condition].copy()` — filter karke ek naya, independent dataframe banana (`.copy()` na lagaayein toh pandas warning deta hai ki 'ye original ka ek view hai ya copy, pata nahi'). `df["new_col"] = value` — ek scalar value poore column mein bharna, agar wo column exist nahi karta toh naya bana deta hai.

EXAMPLE — Control Villages Ki History — Contamination Se Zero Tak

`border_optics_control_results.csv` jo file yahan padhi ja rahi hai, uske peeche ek genuinely dramatic kahani hai. Development Log Entry 23 mein pata chala ki purani control-village list (735 villages) mein se 20 villages, asal mein, 21 treated villages ke hi the — same exact GPS coordinate, sirf alag naam ya script mein likhe hue (jaise Sibia vs Sebia, ya Nisuk vs Nissuk — dono transliteration variants — aur do sabse extreme cases mein toh OSM ka alt-language-name tag hi ek bilkul alag script mein tha: ek village 'Chate' ka Chinese-script naam, aur ek village 'Gunji' ka Devanagari-script naam — dono cases mein Latin-alphabet naam-matching ka toh sawaal hi nahi uthta). `select_control_villages.py` ka exclusion-check sirf naam match karta tha — agar naam hi alag script mein ho, toh match kabhi nahi hota, aur wahi village control list mein bhi ghus jaata, treated list mein bhi. 3 aur villages the jo VVP-I ki official list mein the lekin kabhi geocode nahi ho paaye, isliye unka naam bhi kabhi treated-names set mein nahi aaya — wo bhi control list mein reh gaye apne asli, VVP-listed naam ke saath. Total 21 treated villages ka contamination, 735 mein se 2.9%. Entry 24 mein iska fix likha gaya — ab naam ki jagah, ya naam ke saath-saath, GPS-coordinate-proximity (50m ke andar koi bhi treated village se) aur puri official VVP-I priority-village list (sirf geocoded 251 nahi, saari 552) dono check hoti hain. Entry 25 mein ye fix live Overpass/Nominatim/Earth-Engine access wale machine par asal mein chalaaya gaya — naya control list ban gaya, 732 villages, aur is baar contamination genuinely zero verify hui, sirf 'ceiling' ke neeche nahi. Baad mein Entry 37 mein ek alag issue (district-boundary verification, coordinate-contamination se bilkul alag) fix hua toh list 721 tak aayi — jo aaj is file mein use ho rahi hai. Yahi wo $735 \rightarrow 732 \rightarrow 721$ safar hai jiska docstring mein sirf final '721' number dikh raha tha.

```
cols = ["village_id", "village", "district", "state", "treatment",
        "ndbi_before", "ndbi_after", "lights_before", "lights_after"]
wide = pd.concat([treated[cols], control[cols]], ignore_index=True)
```

Purpose: Sirf zaroori columns choose kiye jaate hain (dono dataframes se), fir treated aur control ko ek dataframe mein jod diya jaata hai — $251 + 721 = 972$ rows ka ek combined table, jisme har row ek village hai (before AND after dono NDBI/lights

columns saath-saath, side-by-side) — isi ko 'wide' format kehte hain, kyunki ek hi row mein multiple time-periods ke columns paas-paas baithe hain. Syntax: `pd.concat([df1, df2], ignore_index=True)` — do dataframes ko upar-neeche stack karna (rows jodna, columns nahi — dhyan dein, PDF 11 mein `axis=1` tha jo columns jodta tha, yahan default `axis=0` hai jo rows jodta hai). `ignore_index=True` — purane dono dataframes ke apne-apne 0,1,2... index bhool kar, naya combined 0 se shuru hota index banao.

```
before = wide.rename(columns={"ndbi_before": "ndbi", "lights_before": "lights"})[
    ["village_id", "district", "state", "treatment", "ndbi", "lights"]]
before["post"] = 0
after = wide.rename(columns={"ndbi_after": "ndbi", "lights_after": "lights"})[
    ["village_id", "district", "state", "treatment", "ndbi", "lights"]]
after["post"] = 1

long = pd.concat([before, after], ignore_index=True)
long["did_term"] = long["treatment"] * long["post"]
return long
```

Purpose: Ye hai wo asli 'wide se long' transformation. wide table ka ek copy banaaya jaata hai jisme `ndbi_before` column ka naam badal kar sirf `ndbi` kar diya jaata hai, aur ek naya column `post = 0` add hota hai (matlab 'ye before-period ka row hai'). Fir wahi cheez `ndbi_after` ke saath, `post = 1` ke saath ('ye after-period ka row hai'). In dono ko phir se `pd.concat` se upar-neeche jod diya jaata hai — ab har village do baar aata hai (ek row before ke liye, ek after ke liye), aur ek hi `ndbi` column hai jiski value `post` ke hisaab se badalti hai. Isi ko 'long' format kehte hain — kam columns, zyada rows, time ek column ban gaya hai (row-level attribute), row-header nahi. Statsmodels ko yahi shape chahiye hoti hai regression chalane ke liye. Aakhri line sabse important hai: `did_term = treatment * post` — ye ek 0/1 column hai jo sirf tab 1 hoti hai jab village treated bhi ho AUR row after-period ka bhi ho. Yahi wo 'interaction term' hai jiska coefficient, regression chalne ke baad, DiD ka asli answer dega. Syntax: `.rename(columns={"old": "new"})` — dictionary se column-naam badalna. `df[list_of_cols]` — sirf wahi columns select karna, order bhi wahi jo list mein diya. Do baar `pd.concat` chalti hai is function mein — pehli baar treated+control jodne ke liye (rows), doosri baar before+after jodne ke liye (rows, phir se).

EXAMPLE — Kyun 'Interaction Term' Hi DiD Ka Jaadu Hai

Socho ek 2x2 grid: treated-before, treated-after, control-before, control-after — chaar groups. Agar hum sirf treatment aur post ko alag-alag regression mein daalein (interaction ke bina), toh model sirf ye keh sakta hai 'treated villages ka average control se X zyada hai' aur 'after period ka average before se Y zyada hai' — dono baatein poore data par average hoke. Lekin humein chahiye ek teesri baat: 'treated villages ka before-se-after badlaav, control villages ke before-se-after badlaav se kitna alag hai?' Yahi `did_term` ka coefficient batata hai — kyunki ye sirf

treated-AND-after row par 1 hai, baaki teen combinations (treated-before, control-before, control-after) par 0. Regression is coefficient ko 'jo bhi extra change treated-after cell mein hai, jo treatment ke akele effect (control ke through) aur post ke akele effect (before ke through) se explain nahi hota' — bilkul wahi jo DiD define karta hai.

Part C — run_did(): Asli Regression Aur Doosra Bada Reversal

```
def run_did(long, outcome, window):
    valid = long.dropna(subset=[outcome]).copy()

    formula_fe = f"{outcome} ~ treatment + post + did_term + C(district)"
    formula_nofe = f"{outcome} ~ treatment + post + did_term"
```

Purpose: Pehli line missing-data wali rows hata deti hai — PDF 11 mein dropna() pattern dekha tha, wahi yahan bhi, is baar ndbi ya lights column par (jo bhi outcome pass kiya jaaye). Fir do formula-strings banti hain: formula_fe mein C(district) hai (matlab 'district fixed effects' — har district ko apna alag baseline-level do), formula_nofe mein nahi (sabhi districts ka ek hi baseline maan lo). Dono model isi liye fit kiye jaate hain taaki result district-composition-sensitive hai ya nahi, dono tarah dikh sake — primary specification FE wali hai, doosri sirf comparison ke liye. Syntax: f-string ke andar {outcome} — variable ki value formula-string ke andar directly bithaana, taaki ek hi function "ndbi" aur "lights" dono outcomes ke liye kaam kare.

EXAMPLE — Fixed Effects Ka Matlab, Ek Chhoti Kahani Se

Socho, Tawang district ke sabhi villages ka baseline NDBI, kisi wajah se (altitude, soil-type, existing urbanization), Kurung Kumey se thoda alag hai — VVP-I se koi lena-dena nahi, bas wahi districts aise hain. Agar model ye 'district difference' na jaanta ho, toh agar treated villages zyada Tawang mein hon aur control villages zyada Kurung Kumey mein, toh DiD coefficient galti se district-difference ko bhi treatment-effect samajh sakta hai. C(district) ye keh kar isse bachaata hai: 'har district ko apna alag intercept do, aur did_term ka coefficient sirf within-district variation se nikaalo' — jitna already district-level difference se explain ho sakta hai, wo alag rakh diya jaata hai, sirf bacha hua treatment-effect measure hota hai.

```
# primary: district FE, clustered SE by district (14 clusters - thin, usual caveat)
model_fe = smf.ols(formula_fe, data=valid).fit(
    cov_type="cluster", cov_kwds={"groups": valid["district"]}
)
# comparison: no FE, HC3 robust SE
model_nofe = smf.ols(formula_nofe, data=valid).fit(cov_type="HC3")
```

Purpose: `smf.ols(formula, data=valid)` ek OLS (Ordinary Least Squares — seedha-saadha linear regression) model define karta hai, aur `.fit(...)` use asal mein data par chala kar coefficients nikaalta hai. Primary model mein `cov_type="cluster"` aur `cov_kwds={"groups": valid["district"]}` ka matlab hai — standard errors (kitna uncertain hai coefficient) ko district ke hisaab se 'cluster' karo, matlab yeh maan kar chalo ki ek hi district ke andar ke villages ek-doesre se correlated ho sakte hain (jaise ek district mein ek hi mahine ka satellite pass, ya ek hi local geography), independent observations nahi. Comparison model mein `cov_type="HC3"` — ye ek alag tarah ka 'robust' standard-error hai jo heteroskedasticity (variance har jagah same nahi hai) ke against protect karta hai, lekin clustering assume nahi karta. Syntax: `smf.ols(...)` — ek function-call jo model-object return karta hai (abhi fit nahi hua). `.fit(cov_type=..., cov_kwds=...)` — method-chaining, seedhe uske upar `.fit()` call karke asal regression chalaana, keyword arguments se SE-type customize karna.

EXAMPLE — Sirf 14 Clusters — Ek Honest Caveat, Comment Mein Hi Likhi

Code ke comment mein hi likha hai: '14 clusters - thin, usual caveat'. Cluster-robust standard errors ka ek known statistical issue hai — jitne kam clusters hon (yahan sirf 14 districts), utna hi in standard errors ka estimate khud thoda unstable ho sakta hai (asymptotic theory bade-N ke liye best kaam karti hai, 14 utna bada nahi hai). Isi wajah se aage PDF 14 mein ek poori robustness-check file hai — `wild_cluster_bootstrap.py` — jo bilkul isi 14-cluster-thin-hone wale concern ko ek zyada rigorous tareeke se test karti hai (ANALYSIS_FREEZE.md ke mutabik, sabhi chaar H4 results wahan bhi null reh jaate hain — is result ko aur confidence milta hai). Abhi ke liye, is file mein ye caveat khud comment mein disclose kar di gayi hai, chhupaayi nahi gayi.

```
n_treated = valid[valid.treatment == 1]["village_id"].nunique()
n_control = valid[valid.treatment == 0]["village_id"].nunique()

coef, se, p = (model_fe.params["did_term"], model_fe.bse["did_term"],
model_fe.pvalues["did_term"])
ci = model_fe.conf_int().loc["did_term"]
coef2, p2 = model_nofe.params["did_term"], model_nofe.pvalues["did_term"]
```

Purpose: Sample sizes nikaale jaate hain — kitne unique treated aur control villages hain valid data mein. Fir fitted model se sirf `did_term` ke stats nikaale jaate hain: `.params` (coefficient ki value), `.bse` (standard error — 'b-se' matlab beta ka SE), `.pvalues` (statistical significance). `.conf_int()` poore model ke saare coefficients ka 95% confidence-interval table deta hai — usme se `.loc["did_term"]` se sirf `did_term` wali row nikaali jaati hai. Syntax: `valid.treatment == 1` — boolean filter, dot-notation se column access (`valid["treatment"]` jaisa hi). `.nunique()` — kitni alag-alag values hain, count karta hai. `model.params["name"]/.bse["name"]/.pvalues["name"]` — ye teeno statsmodels ke fitted-model-object ke Series attributes hain, coefficient-naam se index kiye jaate hain. `.conf_int().loc["name"]` — ek dataframe se, row-label se, ek poori row

nikaalna.

```
print(f"\n--- {outcome.upper()} DiD, window={window} ---")
print(f"n = {len(valid)} village-period observations "
      f"({n_treated} treated villages / {n_control} control villages)")
print("[Primary: district fixed effects, cluster-robust SE by district, 14 clusters]")
print(f"did_term = {coef:+.5f} SE={se:.5f} 95% CI [{ci[0]:+.5f}, {ci[1]:+.5f}] p={p:.5f}")
print("[Comparison: no fixed effects, HC3 heteroskedasticity-robust SE]")
print(f"did_term = {coef2:+.5f} p={p2:.5f}")
```

Purpose: Sirf print statements — dono model ke results readable format mein console par likhe jaate hain, taaki jab script chale, turant dikhe kya nikla. Syntax: {coef:+.5f} — f-string formatting, + ka matlab hai positive numbers ke aage bhi + sign dikhao (na ki sirf negative ke aage -), .5f matlab 5 decimal places tak.

Yahi Hai Wo Line Jahan Doosra Bada Reversal Chhupa Tha

Upar jo run_did() function dikha, wahi function Entry 22 ke baad se chalta aa raha hai, bina khud badle — lekin uska OUTPUT teen baar badla hai, kyunki jo data (long, jo build_panel() se aata hai) ismein jaata hai, wo control-list ke fix hone se pehle aur baad mein bilkul alag tha. Poori kahani step-by-step:

EXAMPLE — Step 1 — Entry 22 Ke Baad, Ek Result Significant Tha (Contaminated 735-List Par)

Jab Entry 22 mein treated aur control dono groups fresh re-extract hue the (PDF 11 mein already dekha), tab is study ka ek control-group result significant nikla tha — full-year night-lights DiD: coefficient +0.1653, 95% CI [+0.0105, +0.3202], $p = 0.0364$. Iske saath do supporting checks bhi the jo isko 'real but fragile' feel karaate the: leave-one-district-out mein 14 mein se 10 baar significant reh jaata tha, aur randomization inference (2,000 permutations) mein $p = 0.0155$ tha. Poore paper mein, is ek result ko 'humara ek surviving significant finding' ki tarah treat kiya ja raha tha.

EXAMPLE — Step 2 — Entry 23: Pata Chala Ki Control List Contaminated Thi

Ek methodology audit mein pata chala ki 735-village control list mein se 20 villages, GPS coordinate se, exact wahi jagah par thay jahan 21 treated villages thay — bas alag naam ya alag script mein likhe hue (jaise upar build_panel() ke EXAMPLE mein dekha). 3 aur villages VVP-I ki official list mein the lekin kabhi geocode nahi hue, isliye unka naam bhi kabhi treated-list mein register nahi hua tha — wo bhi control list mein reh gaye apne asli VVP-listed naam se. Ye contamination sirf control-group ko affect karta hai (H1, H3, buffer-sweep, multi-year trend sab isse untouched hain, kyunki wo control data use hi nahi karte) — lekin H4 DiD iska seedha shikaar tha.

EXAMPLE — Step 3 — Entry 24 Mein Fix Likha Gaya, Entry 25 Mein Live Chalaaya Gaya

select_control_villages.py (PDF 7) mein do fixes likhe gaye: geodesic coordinate-proximity check (koi bhi candidate jo kisi treated village ke 50 meter ke

andar ho, chhod do — naam ki parwaah kiye bina) aur poori 552-naam wali official VVP-I list ke against ek check (sirf geocoded 251 nahi). Ye fix pehle sirf code mein tha (Entry 24), fir asal mein ek live-Overpass-access wale machine par chalaaya gaya (Entry 25) — naya control list bana, 732 villages, aur is baar contamination genuinely verify hui zero (20 se 0, 3 se 0) — sirf ek 'ceiling' assumption ke neeche nahi, seedha directly count karke.

EXAMPLE — Step 4 — Ye Rahi Reversal Table, Wahi Number Jo Ab is Function Se Nikalta Hai

Har check jo pehle is result ko 'real but fragile' bata raha tha, fix ke baad palat gaya:

Check	Pehle (contaminated 735-list)	Baad (clean 732-list, Entry 25)
Primary DiD (FE, cluster SE)	coef +0.1653, p=0.0364	coef +0.05046, p=0.30946
Comparison DiD (no FE, HC3)	p=0.1033	p=0.57800
Leave-one-district-out	10/14 significant	0/14 significant
Randomization inference	p=0.0155	p=0.12994
Log1p-transformed DiD	p=0.0364 (significant)	p=0.50048 (null)
2021 baseline balance	p<0.00001 (imbalanced)	p=0.08976 (balanced)

Chaaron supporting-checks — LOO, RI, log1p, baseline-balance — sab ek saath palat gaye, exactly usi direction mein jo bataata hai ki purana result ek genuine-lekin-fragile effect nahi tha, balki ek contaminated comparison ka artifact tha: jab 'control' group ka kuch hissa asal mein treated group ka hi apna data tha, dobara counted, ulti taraf, usi regression mein.

EXAMPLE — Step 5 — Ek Aur Twist, Entry 37 Mein: 732 Se 721

Baad mein, ek bilkul alag issue pakda gaya — control-village district-boundary verification (Nominatim se real polygon check, sirf bounding-box nahi) 219 mein se 732 villages ke liye fail ho raha tha, kyunki ek network-retry logic missing thi. Fix hone ke baad, control list 732 se 721 tak aaya (Sikkim ke North district ki 93 villages abhi bhi polygon-verify nahi ho paatin — ek genuine data-gap, bug nahi). Is baar H4 DiD ka result khud nahi badla — sabhi chaar combinations null hi rahe — lekin ek connected number ulta palat gaya: full-year lights ka baseline-balance check, jo Entry 25 ke baad p=0.090 (balanced) tha, Entry 37 ke baad phir se p=0.03558 (imbalanced) ho gaya. Interesting baat ye hai ki iss baar reversal 'significant se null' nahi tha, balki ulta — is poore project ki history mein pehli baar ek fix ne kisi cheez ko null se wapas significant banaaya, DiD result khud ko chhue bina. Isko aage `baseline_balance()` function ke saath detail mein dekhenge.

Part D — `baseline_balance()`: Ek Naya Test, Mann-Whitney U

```
def baseline_balance(long, outcome, window):
    # 2021 baseline level check - stand-in for a pre-trends placebo test, which
    # needs a multi-period pre-treatment panel the control group doesn't have
    pre = long[long["post"] == 0].dropna(subset=[outcome])
    t = pre[pre.treatment == 1][outcome]
    c = pre[pre.treatment == 0][outcome]
    u_stat, p = stats.mannwhitneyu(t, c, alternative="two-sided")
```

Purpose: Comment khud bata deta hai iska purpose: ye ek 'stand-in' hai ek asli pre-trends placebo test ke liye (jo PDF 15 mein aayega, jab control group ke paas 2019 ka bhi data hoga). Abhi ke liye, sirf ye check hota hai — 2021 mein (before-period), treated villages ka NDBI/lights, control villages ke NDBI/lights se statistically kitna alag tha? Agar bahut alag tha, toh DiD model ka 'parallel trends' assumption (dono groups similar starting-point se shuru hue) thoda kamzor pad jaata hai — isliye ye ek honest disclosure hai, model ka result invalidate nahi karta, sirf uski ek limitation batata hai. Syntax: `long["post"] == 0` — sirf before-period ke rows chunna. `pre[pre.treatment == 1][outcome]` — do baar filtering, ek hi line mein: pehle treated rows chuno, fir unme se sirf outcome column.

EXAMPLE — Mann-Whitney U — Wilcoxon Ka Cousin, Lekin Paired Nahi

PDF 11 mein humne `stats.wilcoxon()` dekha tha — wo test 'paired' data ke liye hai, matlab har observation ka ek exact partner hota hai (usi village ka before aur usi village ka after). Yahan situation alag hai: treated villages aur control villages do bilkul alag samples hain, koi ek-dosre se 'paired' nahi (village A treated hai, uska koi ek-se-ek control partner nahi hai jiske saath compare kiya jaaye). Iske liye `stats.mannwhitneyu()` use hota hai — ye check karta hai ki do independent samples (yahan treated ka before-NDBI, aur control ka before-NDBI) ek hi distribution se aaye lagte hain ya nahi, bina ye maane ki data normally-distributed hai (jaise ek t-test maanta). `alternative="two-sided"` ka matlab hai — hum sirf ye poochh rahe hain 'kya alag hai', 'kis taraf alag hai' nahi (PDF 11 ke Wilcoxon test mein `alternative="greater"` tha, kyunki wahan hum specifically 'increase hua ya nahi' poochh rahe the — yahan sirf 'balanced hai ya nahi', dono taraf ho sakta hai).

```
print(f"\n--- Baseline (2021) balance check, {outcome}, window={window} ---")
print(f"treated mean={t.mean():.5f} (n={len(t)}) control mean={c.mean():.5f} (n={len(c)})")
print(f"Mann-Whitney U={u_stat:.1f}, p={p:.5f}")
return {
    "outcome": outcome, "window": window,
    "baseline_treated_mean": t.mean(), "baseline_control_mean": c.mean(),
    "baseline_n_treated": len(t), "baseline_n_control": len(c),
    "baseline_balance_p": p,
}
```

Purpose: Result print hota hai, fir ek dictionary return hoti hai jo baad mein JSON summary mein jaayegi. **Syntax:** Dictionary return karna, taaki caller function isse aage collect kar sake — wahi pattern jo PDF 11 ke PS()-jaisa helper function istemaal karta tha.

EXAMPLE — Entry 37 Ka Twist — Yahi Function, Yahi Line, Do Alag Answers

Ye function bilkul wahi hai jo Entry 25 aur Entry 37 dono baar chala — sirf input data (long, jo control-list ke fix se badalta hai) alag tha. Poore study mein current, final numbers ye hain (border_optics_did_summary_fullyear.json/_summer.json se seedha, 721-village list, jo aaj is file se generate hoti hai):

Window	Outcome	Treated Mean	Control Mean	p-value	Verdict
Full-year	NDBI	-0.18459	-0.18327	0.18118	Balanced
Full-year	Lights	0.42821	0.43493	0.03558	Imbalanced
Summer	NDBI	-0.22343	-0.23008	0.46099	Balanced
Summer	Lights	0.40169	0.40717	0.08767	Balanced (borderline)

Full-year lights ka imbalance — yahi wo number hai jo Entry 25 ke baad $p=0.090$ (balanced) tha, aur Entry 37 ke baad $p=0.03558$ (imbalanced) ho gaya. Zaroori baat: is imbalance ka matlab ye NAHI hai ki full-year lights DiD result (jo apne aap mein null hai, $p=0.45334$) galat hai — DiD apna hisaab khud independently karta hai. Ye sirf ek honest disclosure hai ki us particular null result ke peeche ka baseline utna 'clean-matched' nahi hai jitna thodi der ke liye lag raha tha — ek fix jo district-labels ko sahi karne ke liye kiya gaya (treatment-effect se koi lena dena nahi), usne ye surface kar diya. Isko chhupaaya nahi gaya — seedha paper ke §6.7 aur §4.6 mein report kiya gaya hai.

Part E — per_district_summary(): Pooled Result Ke Peeche Ki Kahani

```
def per_district_summary(long, outcome, window):
    print(f"\n--- Per-district treated-vs-control mean change, {outcome}, window={window} ---")
    rows = []
    for district, grp in long.groupby("district"):
        t_wide = grp[grp.treatment == 1].pivot_table(index="village_id", columns="post",
            values=outcome)
        c_wide = grp[grp.treatment == 0].pivot_table(index="village_id", columns="post",
            values=outcome)
```

Purpose: Har district ke liye alag-alag loop chalta hai. `.pivot_table(index="village_id", columns="post", values=outcome)` ek naya chhota table banata hai jisme har row ek village hai, aur do columns hain — 0 (before value) aur 1 (after value) — yaani wapas long format se thodi der ke liye wide format mein, sirf is district ke andar, taaki before-after subtract karna aasan ho. Syntax: `for group_key, group_df in df.groupby("col")` — pandas ka groupby, har unique value ke liye ek (key, sub-dataframe) pair deta hai, loop se iterate kiya ja sakta hai. `.pivot_table(index=, columns=, values=)` — long data ko wide shape mein reshape karna, ek column ki values ko naye columns bana kar.

```
if 0 not in t_wide.columns or 1 not in t_wide.columns or 0 not in c_wide.columns or 1
not in c_wide.columns:
    continue
t_valid = t_wide.dropna()
c_valid = c_wide.dropna()
if t_valid.empty or c_valid.empty:
    continue
```

Purpose: Ye ek defensive check hai — agar kisi district mein saare treated villages ka sirf before-data ho (after ka na ho, ya vice-versa), toh pivot_table mein 0 ya 1 column bilkul banega hi nahi, aur aage subtract karne ki koshish crash kar degi. Ye check usse pehle hi us district ko skip kar deta hai. Fir dropna() se sirf wahi villages rakhe jaate hain jinke paas dono period ka valid data hai — agar koi bhi group khaali reh jaaye, wo district bhi skip. Syntax: `continue` — loop ke current iteration ko yahin chhod kar agle iteration par chala jaana (jaise 'agla district try karo'). or se chaar conditions ek saath check karna. `.empty` — dataframe mein zero rows hain ya nahi, batata hai.

```
t_change = (t_valid[1] - t_valid[0]).mean()
c_change = (c_valid[1] - c_valid[0]).mean()
print(f" {district:16s} treated n={len(t_valid):3d} Δ={t_change:+.5f} "
      f"control n={len(c_valid):3d} Δ={c_change:+.5f} gap={t_change - c_change:+.5f}")
rows.append({
    "district": district, "outcome": outcome, "window": window,
    "n_treated": len(t_valid), "n_control": len(c_valid),
    "treated_change": t_change, "control_change": c_change,
    "gap": t_change - c_change,
})
return rows
```

Purpose: `t_valid[1] - t_valid[0]` — column 1 (after) minus column 0 (before), har village ke liye, fir `.mean()` se poore district ka average change. Yahi cheez control ke liye bhi. Print hota hai, aur ek dictionary rows list mein add hoti hai — poori list function ke end mein return hoti hai. Syntax: `{district:16s}` — f-string alignment, district-naam ko 16-character-chaudi field mein left-align karke print karo (columns seedhi dikhne ke liye). Unicode escape `lu0394` Greek letter Delta (change ka symbol) print karta hai.

EXAMPLE — Ek Pooled Null Ke Peeche Chhupi Heterogeneity — Chamoli Ka Example

Ye function poore study mein ek genuinely useful sanity-check hai, kyunki pooled DiD coefficient (jaise full-year lights ka +0.03460, null) sirf ek AVERAGE hai — individual districts mein kaafi alag-alag kahaniyan ho sakti hain jo average mein cancel ho jaati hain. Real data se ek example: full-year lights ke liye, Chamoli district mein treated villages ka average change +1.18333 hai (control ka sirf +0.11507) — ek bahut bada gap, +1.06823 — jabki North district mein ulta hai, treated +0.02010 vs control +0.11136, gap -0.09126 (control zyada badha!). Ye do districts ek-doesre ko roughly cancel kar dete hain jab poore-14-district pooled model chalta hai. Chamoli ka bada number khud koi 'proof' nahi hai VVP-I ke effect ka — ho sakta hai sirf 8 treated villages hone se ek outlier village ka bada asar pad gaya ho — lekin ye batata hai ki ek single pooled p-value, poori kahani nahi hoti; per-district breakdown hamesha dekhne layak hoti hai.

Part F — main(): Sab Kuch Jodna

```
def main():
    parser = argparse.ArgumentParser(description="Village-level DiD: VVP-I treated vs.
district-restricted non-VVP control.")
    parser.add_argument("--window", choices=["full_year", "summer"], required=True)
    args = parser.parse_args()
    window = args.window

    long = build_panel(window)
    long.to_csv(PANEL_OUT[window], index=False)
    print(f"Panel saved to {PANEL_OUT[window]} ({len(long)} rows)")
```

Purpose: Familiar argparse pattern (PDF 11 mein dekha). build_panel() call hoke wo poora wide-se-long transformation chalti hai jo Part B mein dekhi, aur naya long panel disk par bhi save ho jaata hai — taaki koi aur script (ya khud Sakshi) chaahe toh is intermediate panel ko seedhe padh sake, poori pipeline dobara chalaaye bina. Syntax: Ab tak dekha hua sabse familiar block — argparse setup, function-call, .to_csv().

```
did_results, balance_results, district_rows = [], [], []
for outcome in ["ndbi", "lights"]:
    did_results.append(run_did(long, outcome, window))
    balance_results.append(baseline_balance(long, outcome, window))
    district_rows.extend(per_district_summary(long, outcome, window))
```

Purpose: Teen khaali lists banti hain, fir ek loop dono outcomes ("ndbi" aur "lights") ke liye teeno function chalata hai — matlab poori file sirf ek baar likhe gaye code se, chaar combinations (2 window x is loop ke 2 outcome) ka kaam kar deti hai, jab dono windows ke liye script alag se chalaaya jaaye. Syntax: a, b, c = [], [], [] — ek line mein teen khaali lists banana. list.append(x) — ek item jodna (ek result-dictionary har baar). list.extend(other_list) — append se alag, ye poori doosri list ke items ko current list

mein khol kar jod deta hai (kyunki per_district_summary khud ek list of dictionaries return karta hai, use nested list se bachne ke liye extend chahiye, append nahi).

```
summary = {"did": did_results, "baseline_balance": balance_results}
with open(SUMMARY_OUT>window], "w") as f:
    json.dump(summary, f, indent=2)
print(f"\nSummary saved to {SUMMARY_OUT>window}")

pd.DataFrame(district_rows).to_csv(DISTRICT_OUT>window], index=False)
print(f"Per-district breakdown saved to {DISTRICT_OUT>window}")

if __name__ == "__main__":
    main()
```

Purpose: Saari DiD aur baseline-balance results ek dictionary mein jama karke JSON file mein save ho jaati hain — yahi border_optics_did_summary_fullyear.json/_summer.json files hain jinse is PDF ke sabhi 'Final State' number seedhe liye gaye hain. Per-district rows ek dataframe bana kar apni CSV mein save hoti hain. Aakhir mein familiar if __name__ == "__main__": main() guard. Syntax: with open(path, "w") as f: — ek file ko likhne ke mode mein khola, block khatam hote hi apne aap band ho jaati hai. json.dump(dict, f, indent=2) — Python dictionary ko file mein JSON text ki tarah likhna, indent=2 se human-readable formatting.

Final State — H4 Ka Current, Corrected Verdict

Ye hai is file ka current, 100%-corrected output — Entry 37 ke district-verification fix ke baad, 721-village control list par, seedha border_optics_did_summary_fullyear.json/_summer.json files se liya gaya:

Window	Outcome	DiD Coef	Cluster-SE p	HC3 (no-FE) p	Verdict
Full-year	NDBI	+0.00668	0.29102	0.61263	Null
Full-year	Lights	+0.03460	0.45334	0.70730	Null
Summer	NDBI	-0.00275	0.53098	0.69943	Null
Summer	Lights	+0.01798	0.64393	0.70657	Null

Chaaron combinations null hain — koi bhi window, koi bhi outcome, control-group ke against ek significant treatment-effect nahi dikhata, ab jab control group genuinely clean hai (koi contamination nahi, district-labels verified). PDF 14 mein wild-cluster-bootstrap ye chaaron numbers ek teesri, computationally-heaviest tarike

se dobara test karega (14-cluster concern ko seedha address karte hue) — aur ANALYSIS_FREEZE.md ke mutabik, wahan bhi sab null reh jaate hain. Ye is study ke H1 (PDF 11) aur ab H4, dono ke ek jaisa, consistent null verdict tak pahunchne ki poori journey hai — dono baar, ek genuine data-quality bug fix karne ke baad hi asli answer saamne aaya.

Is PDF mein humne dekha: `did_model.py`, wide-format treated+control data ko long-panel format mein badalta hai (before/after ko rows banaakar, post aur did_term columns generate karke) taaki statsmodels ka OLS chala sake; `did_term` ka coefficient hi DiD ka asli jawaab hai; primary specification district-fixed-effects aur cluster-robust SE use karta hai, jabki no-FE/HC3 specification comparison ke liye. Is file ka control-data input — `border_optics_control_results.csv` — kabhi contaminated tha (21 treated villages, alag naam/script mein, control list mein bhi ghus gaye the), jiski wajah se ek result (full-year lights DiD, $p=0.0364$) significant dikh raha tha; Entry 24-25 ke fix ke baad wahi result null ho gaya ($p=0.30946$), aur Entry 37 ke ek alag district-verification fix ne baseline-balance ko ulta significant kar diya bina DiD result ko chhue. `baseline_balance()` Mann-Whitney U test se parallel-trends assumption check karta hai; `per_district_summary()` dikhata hai ki ek pooled null ke peeche district-level heterogeneity chhupi ho sakti hai (jaise Chamoli vs North ka opposite-sign gap).

Agla PDF (PDF 13) H3 border-proximity hypothesis dekhega — buffer aur multi-year sensitivity ke saath (`h3_border_proximity_robustness.py`, `buffer_sensitivity.py`, `multiyear_trend.py`) — jahan is study ka ek naya, genuinely-significant finding (summer NDBI-border-proximity correlation) surface hota hai. Jab bhi tayyar ho, bata dena.